

Revealing the Risk of Hyper-Parameter Leakage in Deep Reinforcement Learning Models

Linkang Du¹, Member, IEEE, Zhikun Zhang², Member, IEEE, Min Chen³, Member, IEEE, Mingyang Sun⁴, Senior Member, IEEE, Shouling Ji⁵, Senior Member, IEEE, Peng Cheng⁶, Member, IEEE, Jiming Chen⁷, Fellow, IEEE, Michael Backes⁸, Fellow, IEEE, and Yang Zhang⁹, Member, IEEE

Abstract—Deep reinforcement learning (DRL) has been implemented in various critical applications, including smart grids, traffic management systems, and autonomous vehicles. To safeguard intellectual property and mitigate security vulnerabilities, access to DRL models is typically restricted to a closed-box format. This means that specific details, such as the structure of the policy network and optimization processes, are not openly available to users. It is crucial to determine if hyper-parameters can be inferred from observable states and actions within these models, presenting two primary challenges: 1) limited data available from the closed-box model and 2) intertwined effects of hyper-parameters on the model behavior. Since DRL models exhibit varying behaviors in identical tasks depending on their hyper-parameter configurations, we introduce a novel hyper-parameter inference attack against DRL, named HyperInfer, which allows adversaries to deduce the settings of a closed-box DRL model. In order to fully assess the risk of model hyper-parameter leakage, we design two novel state generation methods that provoke divergent responses from DRL models. We also develop an inference framework to elucidate the relationship between model behavior and hyper-parameter settings. Through comprehensive experiments involving multiple DRL models and environments, we demonstrate that model behaviors can indeed reveal hyper-parameter settings, with inference accuracy surpassing 90% in scenarios such as PPO with CartPole. We also discuss key

findings relevant to practical applications and explore how knowledge of hyper-parameters can facilitate more sophisticated attacks. Lastly, we propose potential defensive strategies to minimize the risk of hyper-parameter leakage in DRL models.

Index Terms—Deep reinforcement learning model, hyper-parameter, inference attack.

I. INTRODUCTION

DEEP reinforcement learning (DRL), integrating with the powerful fitting ability of deep neural networks (DNNs), is a promising machine learning paradigm. A famous example, AlphaGo Zero [61], achieves the human-champion-defeating ability by playing games against itself without human data in three days. DRL has also been applied to many complex decision-making tasks, such as autopilot [13], robot control [1], financial transactions [11], esports [7], [61], etc.

DRL model generally serves as a closed box due to the fact that disclosing hidden details may render the model more susceptible to attacks from adversaries, such as model extraction attacks [5], [9], [56], [87], adversarial policy attacks [6], [28], [34], [38], and backdoor attacks [32], [68], [72], [82]. On the other hand, the architecture and parameters of a DRL model are protected in legal as intellectual property (IP) [69], [81]. Thus, it is crucial to investigate the amount of information gained from closed-box access to a DRL model.

Previous studies [5], [9], [56], [87] mainly target the stealing of functionality, which imitates the behavior of the target model instead of inferring the model's internal information. Oh et al. [49] and Wang et al. [66] have investigated the hyper-parameter inference attack against supervised learning models. However, those techniques cannot be trivially applied to the DRL scenario due to the following challenges. First, in supervised learning scenarios, the adversary can use the training datasets as high-quality query input. However, the adversary has no off-the-shelf query input in DRL, since the DRL models typically learn from the interactions with the environment. Second, supervised learning models often have explicit labeling information for an input, while DRL models may have multiple equivalent actions for a state. For example, if the DRL agent finds two paths of the same length in a maze game, then the different actions made at the first bifurcation are equivalent. Finally, the hyper-parameters have coupling impacts on the states and actions of a DRL model, i.e., when we try to infer a type of hyper-parameter, the other hyper-parameters are the disturbance factors.

Received 30 April 2024; revised 29 July 2025; accepted 2 September 2025. Date of publication 6 October 2025; date of current version 14 January 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB3102100, in part by the National Natural Science Foundation of China under Grant 62293511, Grant 62402379, Grant 62402431, Grant 62441618, Grant 62172243, and Grant 72571007. The work of Min Chen was supported in part by the CiCS project of the Gravitation research program and in part by the Dutch Research Council (NWO) under Grant 024.006.037. The work of Zhikun Zhang was supported in part by Zhejiang University Education Foundation Qizhen Scholar Foundation. (Corresponding authors: Peng Cheng; Zhikun Zhang.)

Linkang Du is with Xi'an Jiaotong University, Xi'an 710049, China (e-mail: linkangd@xjtu.edu.cn).

Zhikun Zhang, Shouling Ji, and Peng Cheng are with Zhejiang University, Hangzhou 310027, China (e-mail: zhikun@zju.edu.cn; sj@zju.edu.cn; lunerheart@zju.edu.cn).

Min Chen is with Vrije Universiteit Amsterdam, 1081 HV Amsterdam, Netherlands (e-mail: m.chen2@vu.nl).

Mingyang Sun is with Peking University, Beijing 100084, China (e-mail: smy@pku.edu.cn).

Jiming Chen is with Zhejiang University, Hangzhou 310027, China, and also with Hangzhou Dianzi University, Hangzhou 310005, China (e-mail: cjm@zju.edu.cn).

Michael Backes and Yang Zhang are with the CISPA Helmholtz Center for Information Security, 66123 Saarbrücken, Germany (e-mail: director@cispa.de; zhang@cispa.de).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TDSC.2025.3611881>, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2025.3611881

Our contributions: To our knowledge, this is the first work to investigate the risk of hyper-parameter leakage in DRL models. The key insight is that the *disagreement actions*, i.e., the DRL models' inconsistent behaviors in some states will expose their inner information. Based on this observation, we first build a series of states to collect the actions of a DRL model, and then leverage a classifier to reveal the architectures and optimization processes of the victim closed-box DRL model based on the states and predicted actions. Since disagreement actions contain more knowledge of hyper-parameter settings, the inference attack will become more effective with a high-quality state sequence, i.e., stimulating more disagreement actions.

Concretely, we propose two state-generation mechanisms to construct state sequences. The *passive* state generation method builds the state sequences based on passive recording of the interaction between the DRL model and environment, which solves the first challenge mentioned above. However, directly obtaining states from the environment only weakly stimulates DRL models to generate disagreement actions. To make agents with the same hyper-parameter settings behave similarly, and agents with different hyper-parameter settings behave differently, we further propose an *active* state generation method. Specifically, besides training the classifier to predict hyper-parameter settings, the active method back-propagates the partial loss of the classifier model and optimizes the states. The optimized states can induce DRL models with different hyper-parameter settings to generate more disagreement actions. We empirically find that the continuously adjusted states slow down the over-fitting of the classifier and further improve the inference accuracy.

Our experimental results show that the inference accuracy of the hyper-parameter activation function exceeds 60% across multiple DRL models and environments, even more than 90% on PPO agents. We further evaluate four representative influential factors that affect the inference performance, i.e., the victim DRL model's reward criterion, the hyper-parameter coupling, the length of state-action sequence, and the amount of shadow DRL models. Based on the experimental results, we summarize three important observations concerning the hyper-parameter leakage of the DRL model. 1) The DRL models with higher reward criteria suffer higher hyper-parameter leakage risk, which warns DRL practitioners since hyper-parameter privacy becomes more vulnerable with the DRL performance improving. 2) The hyper-parameter has various influence strengths on the model action preference. For example, the adversary can easily discriminate activation functions or optimizers from the DRL model's behavior, while it is difficult to distinguish learning rates. 3) The active inference attack method obtains outperforming results with less state generation and DRL model training costs, which dedicates HyperInfer as a highly efficient hyper-parameter inference mechanism.

We take the existing attack against closed-box DRL models as a case study and show the effectiveness improvement of these attacks with the hyper-parameter information, to demonstrate the necessity of exploring the risk of leakage of DRL model hyper-parameters. Finally, we evaluate two potential defenses to reduce the information accessible to the adversary, i.e., only publishing the predicted action instead of confidence values

(posteriors) and adopting differential privacy to perturb the action. However, the inference accuracy is still 20% higher than the baseline with the active inference attack methods in some settings. We hope that this study will raise practitioners' awareness about the privacy threats of DRL models.

In summary, our contributions are three-fold:

- We propose a hyper-parameter inference framework against DRL models, HyperInfer, which indicates the model-level privacy concerns of the current DRL models.
- We demonstrate the effectiveness of HyperInfer on five DRL models and four environments. We also systematically analyze various experimental factors and conclude several important observations about the hyper-parameter leakage in practice.
- We show that the hyper-parameter settings can significantly improve the threat of the existing attacks, which calls additional attention to protect the hyper-parameter.

II. BACKGROUND

A. Reinforcement Learning Problem

The reinforcement learning (RL) model aims to learn an optimal (or nearly-optimal) policy to maximize the “reward function” or other user-provided reinforcement signals during the immediate rewards accumulation process. RL is similar to the processes that occur in animal psychology. For example, suppose that a buzzer or metronome rings before food (reward). A dog tends to associate the sound with food and salivates on the sound stimulus alone [53]. Typically, the RL problem is modeled as a Markov Decision Process (MDP) problem, which consists of four components:

- A state space $\mathbb{S}$ contains all possible states of the model, where $s_t \in \mathbb{S}$ is the state at the time step t .
- An action space $\mathbb{A}$ contains all possible actions of the model, where $a_t \in \mathbb{A}$ is the action at the time step t .
- An environment transition dynamics $\mathcal{T}$ is defined as a probability mapping from state-action pairs to new states $\mathcal{T} : (\mathbb{S} \times \mathbb{A}) \times \mathbb{S} \rightarrow [0, 1]$.
- A reward function $\mathcal{R} : \mathbb{S} \times \mathbb{A} \rightarrow \mathbb{R}$ outputs the expected reward $r_t \in \mathbb{R}$ if the model takes action a_t at state s_t .

The goal of an RL algorithm is to learn a policy π that maps state-action pairs to a probability distribution $\pi : \mathbb{S} \times \mathbb{A} \rightarrow [0, 1]$, to maximize cumulative reward.

$$\pi^* = \arg \max_{\pi} \sum_{t=t_0}^T \sum_{a_t \in \mathbb{A}} \gamma^{t-t_0} \mathcal{R}(s_t, a_t) \pi(s_t, a_t),$$

where the discount factor γ is applied to discount future rewards in the accumulated reward. T is the terminal time step of one round of games and t_0 is the initial time.

Example: Fig. 1 gives a running example, where the DRL model plays with the flappy bird game.¹ The state includes the positions of the bird and the obstacles. The DRL model selects whether to click or not to click (action) to let the bird fly up or down. The reward shows the total score obtained by the DRL

¹<https://flappybird.io>

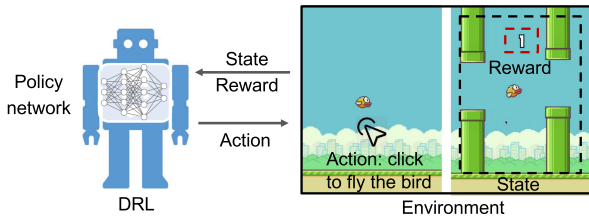


Fig. 1. An example of the reinforcement learning problem.

model. The DRL model obtains the state and reward from the environment and then selects an action from the motion space at each time step. Then, the environment moves to a new state and updates the reward. Based on the collected state, action, and reward, the DRL model trains a policy (neural networks) to maximize the total expected reward, e.g., the total score at the end of the game.

B. Deep Reinforcement Learning Model

The goal of RL is to find an optimal behavior strategy for the model to obtain optimal rewards. From the perspective of optimization strategy, existing RL models can be categorized into three classes [3], [71].

Value-based RL models: Q-learning [75] is a primary value-based RL model, which finds the optimal policy by maximizing the expected value of the total reward over successive steps. Q-learning has a function that calculates the quality of a state-action combination: $Q: \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$. For the basic Q-learning, Q is defined as a tabular with random values, i.e., *Q-table*, before learning begins. At each time step t , the model selects an action a_t , receives a reward r_t , enters a new state s_{t+1} (which may depend on both the previous state s_t and the selected action a_t), and then updates the values in Q . The core of the algorithm is as follows.

$$Q^{\text{new}} \leftarrow Q + \alpha \cdot \left(r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right), \quad (1)$$

where $\alpha \in (0, 1]$ is the learning rate and Q^{new} changes more rapidly with larger α . Q-learning uses the weighted average between the old Q values and the new information to conduct a value iteration update as (1).

However, when the state and action space become enormous, the basic Q-learning can hardly maintain and store an extensive Q-table. To overcome this problem, the Deep Q-Learning Network (DQN) [47] integrates the DNNs to approximate the values in the Q-table. Based on the superior capabilities of neural networks to learn high-dimensional feature representations and function approximation properties, DQN is used to play Go [61] and Atari games at the expert level [47].

Policy-based RL models: Unlike approximating the Q-table, the policy-based RL model directly optimizes the action prediction. For example, REINFORCE [76] updates a neural network-based policy by leveraging gradient estimation. Recent policy-based RL models tend to use DNN (e.g., multilayer perceptron [79] or recurrent neural networks [84]) to model the policy π , called policy network. The objective function of

the policy network is defined as the expectation of the total discounted rewards in a game.

$$J(\theta) = \mathbb{E}_{(s_t, a_t) \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]. \quad (2)$$

According to [33], the policy gradient can be written as

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(s, a) v_t],$$

where v_t is an unbiased sample of $Q_{\pi_\theta}(s, a)$ that indicates the long-term action value of the MDP problem, and $\pi(s, a)$ is the policy network.

Actor-Critic RL models: The actor-critic RL model [46] uses two components: actor and critic, to separately characterize the policy and the action-value function.

The actor decides which action should be taken, and the critic scores the actor's action. The actor's learning is based on the policy gradient approach. The critic evaluates the actor's action by computing the action-value function. Since the action-value function can assist the policy update, an actor-critic RL model can reduce the gradient variance and accelerate the convergence process.

Therefore, the actor-critic RL model has two separate objective functions for actor and critic networks. For the actor, the objective function has a similar form to (2), while using $\mathcal{A}_{\pi_\theta}(s, a)$ instead of $Q_{\pi_\theta}(s, a)$.

$$\mathcal{A}_{\pi_\theta}(s, a) = Q_{\pi_\theta}(s, a) - V_{\pi_\theta}(s), \quad (3)$$

where $V_{\pi_\theta}(s)$ represents the average value of state s . When $\mathcal{A}_{\pi_\theta}(s, a) = 0$, there will be no extra reward in tuning the probabilities of the actions, and then the actors' gradients will be set to zero. The objective function of the critic is to minimize the error between the estimated and the true action-value functions. To accelerate the training process of the actor-critic RL models [58], recent research integrates a new objective function called "clipped surrogate objective function" into the actor-critic algorithm, i.e., proximal policy optimization (PPO) [23], [59], which performs comparably or better than state-of-the-art approaches while being much simpler to implement and tune. Due to ease of use, OpenAI has set PPO as its default DRL algorithm.²

In the evaluation, we choose DQN [47], PG [76], and PPO [23], [59] as they are fundamental DRL methods that exemplify value-based, policy-based, and actor-critic approaches, respectively. Furthermore, we incorporate Trust Region Policy Optimization (TRPO) [22] and Soft Actor Critic (SAC) [58] to represent advanced DRL models.

C. Hyper-Parameters in DRL Models

Since the DRL model integrates a DNN model as the policy network, the hyper-parameters in DRL can be categorized into three parts, i.e., the network architecture hyper-parameters of the DNN model, the optimization process hyper-parameters of the DNN model, and the hyper-parameters of the RL model.

In this work, we mainly focus on six DRL model hyper-parameters (Table I), which contain the hyper-parameters in the

²<https://openai.com/blog/openai-baselines-ppo/>

TABLE I
THE HYPER-PARAMETER SETTINGS

Category	Hyper-parameter	Values
Network Architecture	Activation Function	ReLU, ELU, Tanh
	Hidden Layers	2, 3, 4
	Neuron Number	64, 128, 256
Optimization Process	Optimizer	SGD, Adam, RMSprop
	Learning Rate	1e-3, 5e-4, 1e-4
RL model	Gamma	0.9, 0.95, 0.99

previous DNN hyper-parameter inference attack [49] and the discount factor γ of an RL model. The selected hyper-parameters are relevant to nearly all deep reinforcement learning (DRL) models. Other hyper-parameters are not considered in this paper, as they may not be part of some DRL models. For example, convolutional layers are typically applied to vision-related tasks, while the learning rate schedule is an optional aspect of DRL model training. We should note that the core idea of HyperInfer, which involves deducing the model's internal parameters from its input and output and optimizing the input to gain further internal details, can be expanded to a vast array of hyper-parameters.

III. PROBLEM STATEMENT

A. Motivation

Amitai et al. [2] show that two DRL models with different hyper-parameter settings have different behavior preferences on the same task. For example, in the autonomous driving environment for the merging task [14], a safety-oriented DRL model prefers to slow down and merge behind the non-autonomous vehicle, decreasing the chances of a collision. In contrast, an aggressive DRL model prefers to speed up and merge in front of the non-autonomous car. Therefore, DRL models with different architectures and optimization processes have separable action tendencies that expose their internal information. In this work, we plan to answer the following two questions.

- Q1. Can an adversary infer the hyper-parameter settings from the behavior of the DRL model?
 Q2. What kind of state is more beneficial to infer the hyper-parameter settings?

B. Threat Model

Attacker's goal: The adversary aims to infer the hyper-parameter settings of a DRL model, such as the architecture (e.g., the type of the non-linear activation function) and optimization process (e.g., the type of the optimizer). The adversary aims to achieve high accuracy in inferring the hyper-parameter settings from the victim DRL model.

Attacker's background knowledge and capability: We consider an adversary who has closed-box access to the victim DRL model. Note that this is the most general and challenging scenario for the adversary. A typical application scenario is that a service provider trains a DRL model and then provides an API to the customers. A malicious customer wants to infer the hyper-parameter settings from the victim DRL model either for

stealing intellectual property, or using them as a stepping stone to launch attacks on other customers' models.

When the victim DRL model interacts with the environment, the adversary can observe its states and actions. In addition, since the victim DRL model is applied in the adversary's environment, the adversary can adjust the environment state within the valid boundary. The attacker can calculate the average cumulative reward to estimate the victim's reward criterion (performance) from the model's interactions with the environment in practice. For example, if the average cumulative reward of a DRL model is 300 after a number of episodes, the model's reward criterion equals 300. The victim's framework can be inferred by Chen et al. [8].

Definition 1: (DRL hyper-parameter inference attacks.) By closed-box access to the victim model, the DRL hyper-parameter inference attack aims to infer the hyper-parameter settings of the victim DRL model based on the state-action records.

IV. HYPER-PARAMETER INFERENCE ATTACK

A. Workflow of HyperInfer

In Fig. 2, we take a flappy bird DRL model as an example to illustrate the workflow of the DRL hyper-parameter inference attack. We divide the workflow of HyperInfer into four steps and describe the steps in detail.

Step 1: Shadow Model Generation (SMG): In the left dashed box of Fig. 2, the adversary uses the flappy bird environment to produce a set of shadow DRL models, which have different hyper-parameter combinations. For example, to infer the activation function of the victim DRL model, the attacker trains the shadow DRL models with different types of activation function.

Step 2: State-action Sequence Collection (SSC): In the second dashed box, the shadow DRL models observe the states of the environment and take actions. At each step, the adversary records the state s_t and the action a_t^n of each shadow DRL model, where a_t^n represents the n th DRL model action at the t th step. Based on the disagreement of models' actions, the attacker can aggregate the hyper-parameter information from the state-action sequences. Intuitively, the adversary will find more divergence of the model behavior with a longer length of the collected state-action sequence. Thus, we also evaluate the efficacy of inference with different lengths of state-action sequences in Appendix D, available online.

Step 3: Attack Model Training (AMT): After the above two steps, the adversary obtains a set of state-action sequences and the corresponding hyper-parameter settings. For all state-action sequences, the adversary concatenates the state and action at each step into a $(|s| + |a|)$ -dimensional feature vector. The feature vectors are stitched along the time dimension to form a $T \times (|s| + |a|)$ -dimensional feature x , and the adversary acquires a feature set X with a hyper-parameter setting label set Y . Then, a supervised classifier can be used to characterize the relationship between the feature X and the label Y . The training objective is

$$\min_{\theta} \mathbb{E}_{x \sim \mathcal{X}} [\mathcal{L}(f_{\theta}^p([x]_{i=1}^n), y^p)], \quad (4)$$

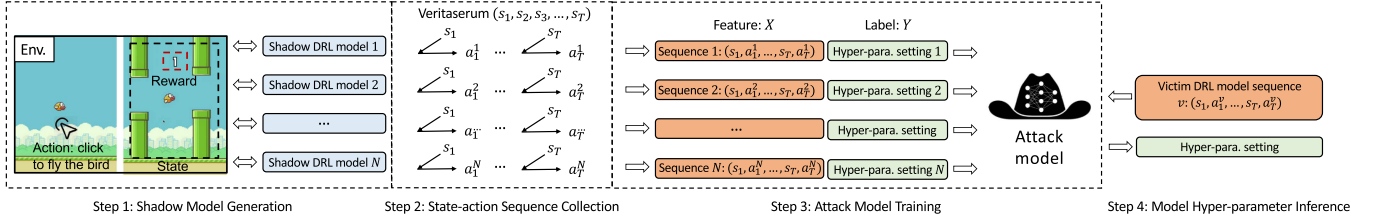


Fig. 2. Workflow of HyperInfer. The four steps in the HyperInfer algorithm, i.e., shadow model generation, state-action sequence collection, attack model training, and model hyper-parameter inference. HyperInfer infers the policy network information from the state and action of the victim DRL model.

Algorithm 1: State Perturbation.

Input: Environment E , state s
Output: State s'

- 1: // Obtain the bound for each dimension of E
- 2: low_bound, high_bound = $E.spaces.Box()$
- 3: // Limit the scope of the perturbation
- 4: low_bound, high_bound = $\frac{low_bound}{10}, \frac{high_bound}{10}$
- 5: **for** i th dimension in $E.spaces$ **do**
- 6: $\sigma = \min(|low_bound|, |high_bound|)$
- 7: **while** True **do**
- 8: // Check the new state in the boundary
- 9: $s'[i] = s[i] + N(0, \sigma^2)$
- 10: **if** $s'[i]$ in $E.spaces.Box(dim=i)$ **then** :
- 11: break
- 12: **end if**
- 13: **end while**
- 14: **end for**
- 15: Return $s' = 0$

where $x \in X$, $y^p \in Y$, f_θ^p represents the attack model, and $\mathcal{L}$ is the cross-entropy loss.

Step 4: Model Hyper-parameter Inference (MHI): In this step, the attacker constructs the state s_t and observes the victim DRL model action a_t^v . After the same pre-processing of the states and the actions as Step 3, the attacker puts the state-action feature sequence into the attack model and then obtains the prediction.

From Step 2 and Step 3, the inference accuracy may vary with different state sequences with the same set of shadow DRL models. For example, if the bird's position is lower than the obstacles, all shadow models tend to behave the same, where the attack model cannot obtain any knowledge from the state-action sequence. In contrast, the inference attack will be more effective if the attacker constructs states that stimulate the DRL models to produce disagreement actions.

Gajcin et al. [14] defined the *preference-based disagreement* in DRL scenarios, which can serve as a criterion to determine whether a state causes disagreement actions between the DRL models. However, the definition is limited in practice. First, the definition is about judging instead of generating a state to cause disagreement actions. In addition, the definition is applied to two DRL model scenes, while there are more shadow DRL models in inference attacks. Thus, we design two mechanisms to build the states in Section IV-B.

B. Attack State Generation

In the attack procedure, the sequence of states plays an important role in exposing the hyper-parameter settings of the victim model. Below, we elaborate on the generation and optimization of the states in detail.

Passive state generation mechanism: To obtain the state sequence, an intuitive way is to collect it directly from the interactions between the shadow DRL models and the environment. Algorithm 2 shows the detailed state generation. The attacker randomly chooses a DRL model with policy π_θ , and then initializes the environment with an arbitrary s_0 in the state space. For example, the bird located at the center is a valid state in the flappy bird game. The DRL model puts s_0 into the policy network π_θ , and then obtains the action a_1 . The environment transfers to the next state s_1 with a symbol to determine whether the game is over. The attacker inserts s_1 into state sequence S . If the *done_signal* is *True*, the s_t is the termination state of the environment and the attacker checks whether the time step t equals to T . If $t < T$, the attacker generates a new initial state s'_0 by Algorithm 1 and re-initializes the environment. Then, the attacker continues the state collection until the time step reaches T . If there are some physical constraints between dimensions, we only need to perturb some of the dimensions using Algorithm 1, and then calculate the rest of the dimensions by the constraints.

The passive method is an efficient strategy to generate states. However, it has two drawbacks in practice: 1) the white-box shadow models are not fully utilized, i.e., not using the gradient information of the white-box models; 2) the attack model is easy to over-fitting on the shadow models due to the static state sequence.

Active state generation mechanism: To overcome the shortcomings in the passive method, the attacker uses the gradients of the shadow DRL models to generate a more effective state sequence, i.e., active state generation method. Since the shadow DRL models are white-box, the attacker can access both their hyper-parameter settings and the gradient information in the policy networks. Compared to Algorithm 2 with a static state sequence, Algorithm 3 updates the state sequence in the attack model training process, i.e., dynamic state sequence. We optimize the states to reduce the classification loss when training the attack model to strengthen the relationship between state-action sequences and hyper-parameters. We tailor the objective under state optimization constraints as

$$\min_S \mathbb{E}_{M \sim \mathcal{M}} [\mathcal{L}(f_\theta^p((S, M(S))), y^p)], \quad (5)$$

Algorithm 2: Passive State Generation Mechanism.

Input: Shadow DRL models M , environment E , the max time step T

Output: State sequence $S = \{s_1, s_2, \dots, s_T\}$

- 1: Randomly select a DRL model with policy π_θ
- 2: Let $S = \{\}$
- 3: Initialize E randomly with s_0
- 4: done_signal = False
- 5: **for** time step t from 1 to T **do**
- 6: **if** done_signal == True **then** :
- 7: $s'_0 = \text{Algorithm 1}(E, s_0)$
- 8: Re-initialize E randomly with s'_0
- 9: // Get the action from the DRL model
- 10: $a_t = \arg \max_{a \in \mathbb{A}} \pi_\theta(s'_0, a)$
- 11: **else**
- 12: $a_t = \arg \max_{a \in \mathbb{A}} \pi_\theta(s_{t-1}, a)$
- 13: **end if**
- 14: // Get the new state s_t and the game over signal
- 15: $s_t, \text{done_signal} = E(a_t)$
- 16: Insert the s_t into S
- 17: **end for**
- 18: Return $S = 0$

where S is the state sequence, $\mathcal{M}$ is the distribution of the DRL shadow models, y^p is the ground truth label of the hyper-parameter p , and $\mathcal{L}$ is the cross-entropy loss. As shown in line 8 of Algorithm 3, the loss of the attack model f_θ^p is back propagated to optimize the state sequence S for every m step in the training process. The attacker rebuilds a new feature set X based on the updated state sequence S . The attacker then trains the classifier with X and Y until the next state update. The attacker uses the latest state sequence to collect the actions from the victim DRL model.

C. Discussion

HyperInfer shares the spirit of the reverse engineering strategy proposed by Oh et al. [49], i.e., a general approach to hyper-parameter extraction attacks against image classifier models, which also 1) infers internal information from a DNN model from a sequence of queries, and 2) generates a more effective input to enhance reversal accuracy against closed-box models. However, HyperInfer differs from the reverse-engineering strategy in several essential aspects. It should be noted that these differences are mainly due to the fact that these two methods work for different kinds of victim models. That is, Oh et al. [49] is for the image classifier models with the DNN framework, while HyperInfer works for the sequential decision-making DRL models.

- *Lack of Off-the-shelf Datasets:* Unlike the DNN scenarios, where Oh et al. [49] can use training samples as high-quality query input, the DRL models learn from the environment's interactions. Thus, HyperInfer has no off-the-shelf query input. We need to construct the query input based on the trained shadow models.

Algorithm 3: Active State Generation Mechanism.

Input: Shadow DRL models M , environment E , time step T , hyper-parameter setting label set Y , attack model f_θ^p

Output: State sequence $S = \{s_1, s_2, \dots, s_T\}$

- 1: // Obtain the initial state sequence
- 2: $S = \text{Algorithm 2}(M, E, T)$
- 3: state_update = False
- 4: **for** epoch i from 1 to n **do**
- 5: **if** $i \% m == 0$ and state_update == False **then** :
- 6: $f_\theta^p.\text{loss} = f_\theta^p.\text{loss_calculation}(X, Y)$
- 7: // Update S value
- 8: Back-propagate $f_\theta^p.\text{loss}$ to optimize S
- 9: state_update = True
- 10: **end if**
- 11: **if** $i == 1$ or state_update == True **then**
- 12: // Collect action sequence from M
- 13: $X = []$
- 14: **for** j th shadow model in M **do**
- 15: $\{a_1, a_2, \dots, a_T\}_j = M[j](S)$
- 16: $X.\text{append}(\text{concatenate}(\{a_1, a_2, \dots, a_T\}_j, S))$
- 17: Convert X to tensor
- 18: **end for**
- 19: state_update = False
- 20: **end if**
- 21: // Update attack model weights
- 22: $f_\theta^p.\text{train}(X, Y)$
- 23: **end for**
- 24: Return $S = 0$

- *Sequential Query Input:* The reverse-engineering strategy [49] crafts a single query input to extract information from the DNN models, i.e., it usually processes single image sample at each step. The DRL models are designed to optimize cumulative rewards; the internal information contained in each step is limited (see Section II); thus, HyperInfer needs to build a sequence of inputs to infer the hyper-parameters.
- *New Hyper-parameter:* Besides the hyper-parameters considered in [49], HyperInfer takes into account the special hyper-parameter of DRLs, i.e., discount factor γ .
- *Non-uniqueness of the Policy Behavior:* In classification tasks, each instance receives one accurate label. On the other hand, RL problems often involve numerous strategies, thereby complicating the inference attack.

V. EVALUATION

In this section, we utilize HyperInfer to reveal the hyper-parameter leakage risk of DRL models. We adopt five DRL models and four environments, which have been widely used in previous DRL studies [8], [16], [51], [68], [78]. Concretely, we aim to answer the following questions.

- Q1. Can an adversary infer the hyper-parameter settings from the behavior of the DRL model?
- Q2. What kind of state is more beneficial for inferring the hyper-parameter settings?

- Q3. How do the other experimental factors, i.e., the victim DRL model's reward criterion, the hyper-parameter coupling, the length of state-action sequence, and the number of shadow DRL models influence the attack?
- Q4. How to mitigate the risk of hyper-parameter leakage?

A. Experimental Setup

Environments: We choose the “CartPole”, “Acrobot”, “Ant”, and “HalfCheetah” environments in the Gym.³ We believe that the environments are representative: 1) Gym is a mainstream platform that provides a unified interface for training and testing the DRL models. 2) Selected environments are commonly used in academia to evaluate the effectiveness of attacks against DRL models [8], [30], [52].

Targeted DRL model: We adopted DQN [47], PG [76], PPO [23], [59], TRPO [22], and SAC [58] models during evaluation. The DQN, PG, and PPO models represent three basic ideas of the RL models, i.e., value-based strategy, policy-based strategy, and actor-critic strategy. Most of the existing model-free RL methods are evolved from these three approaches. We select TRPO and SAC to represent the advanced RL models.

Targeted hyper-parameters: Table I shows the values of hyper-parameters. The first three hyper-parameters are about the architecture of the DRL policy network, and the others are related to the optimization process. For each hyper-parameter, we choose three different values, which are usually the default selection in open-sourced frameworks [12], [24], e.g., ReLU and the Adam optimizer.

Shadow model construction: For each combination, we collect 20 shadow DRL models for each hyper-parameter setting (totally 20×3^6), and then divide the shadow models into a training set (16×3^6) and a test set (4×3^6), which is the default setup for the subsequent experimental steps. Since it is difficult to train a DRL model satisfying the reward criterion under some hyper-parameter values [46], the distributions of hyper-parameter values are possibly not uniform among the shadow DRL models. The distributions of the hyper-parameters are shown in Appendix B, available online.

Evaluation metric: In our experiment, we follow the metric commonly used to evaluate inference attacks, measuring the accuracy of prediction [8], [49], [51]. We utilize shadow models in the training set to generate state-action sequences and train the attack model. Then, we take the shadow models in the test set as the victim model and infer the hyper-parameter settings based on their state-action sequences.

Methods: We compare the inference performance between the passive state generation and the active state generation methods. “passive_env” (Algorithm 2) uses the states directly from the interaction between the DRL model and the environment. “active_env” optimizes the states from “passive_env” by leveraging the state optimization in Algorithm 3. For both methods, the default sequence size is 200 in the experiments. The baseline method, i.e., “majority_vote”, adopts the largest proportions as the prediction for the victim DRL model.

Experimental settings: We adopt the ResNet framework as our attack model and illustrate its architecture in Appendix A, available online. We train the attack model for 400 epochs. For the passive method, we use the SGD optimizer to train the attack model with a learning rate of 0.01. We leverage an additional Adam optimizer with a learning rate of 0.01 to tune the states every 10 epochs during the attack model training.

B. Overall Performance

We show the effectiveness of HyperInfer in six combinations of basic DRL models and Gym environments.

Setup: We use the reward criteria in OpenAI Gym leaderboard⁴ for the victim DRL models, which are 500 for Cartpole and -100 for Acrobot, respectively. In Fig. 3, we vary the types of hyper-parameter on the X -axis and show the corresponding inference accuracy on the Y -axis. Each histogram in the plots shows the average inference accuracy of 3 repeated experiments with the standard deviation.

Observations: We have the following observations from Fig. 3. 1) The inference accuracy of HyperInfer is all higher than the baseline method, which means that the behaviors of DRL models can indeed expose the hyper-parameter settings. For example, the inference accuracy for the activation function is beyond 60% across the experiment settings, even exceeding 90% on PPO.

2) HyperInfer obtains different inference accuracy over various DRL models. The adversary's inference effectiveness on the PG and PPO models is better than that of the DQN models for the hyper-parameters optimizer and learning rate. For the PG and the PPO models, HyperInfer achieves a huge boost in accuracy on the optimizer and the learning rate, while HyperInfer slightly surpasses the baseline method for the DQN model. Recalling Section II-B, the DQN models use the policy network to approximate the Q-table in the training process and choose the action according to the formed Q-table, i.e., indirectly act. PG and PPO use the policy network to directly build the relation between states and actions without the Q-value approximation in DQN, i.e., directly act. Thus, it is more difficult to infer the optimization-related hyper-parameters from DQN.

3) The inference accuracy of HyperInfer varies over the hyper-parameters. In Fig. 3, HyperInfer usually obtains higher inference accuracy on the hyper-parameter activation function compared to the hyper-parameter learning rate. As a structural hyper-parameter, the different activation functions have unique features. For example, the Tanh activation function is symmetric about (0,0), while the ReLU and ELU activation functions are asymmetrical. In addition, the functional features of activation do not change once the DRL model is built. Therefore, the activation function of the DRL model is at increased risk of leakage. The hyper-parameter learning rate controls the update speed of the models' weights. Compared to the activation function, the learning rate guides the model's actions by affecting the model's weight values. Thus, the features of the learning rate become blurred as the training process.

³<https://www.gymnasium.ml/>

⁴<https://github.com/openai/gym/wiki/Leaderboard>

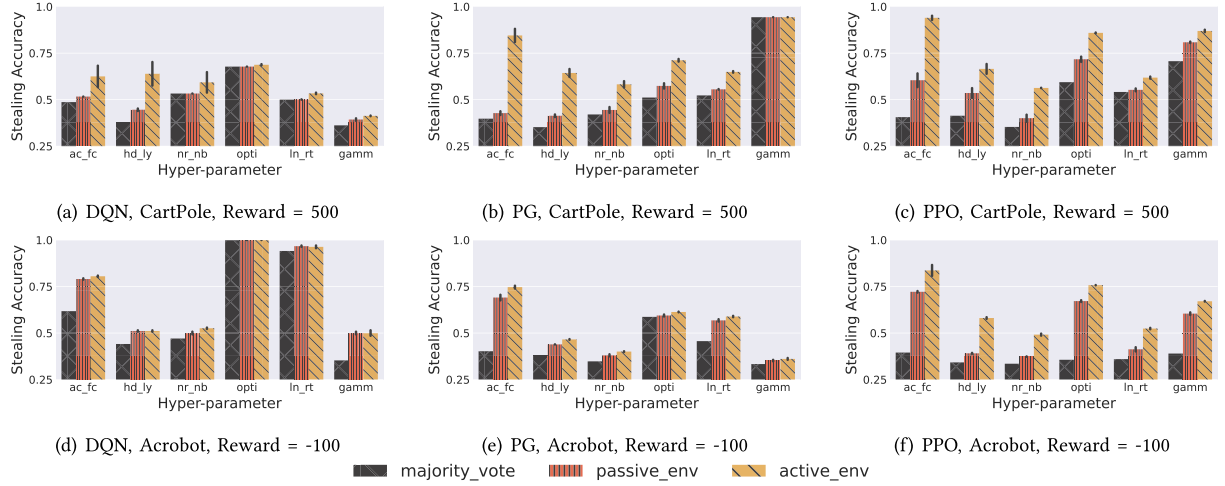


Fig. 3. The inference accuracy of different methods on six combinations of the DRL models and the environments. The x labels of each plot correspond in turn to the hyper-parameters activation function, hidden layers, neuron number, optimizer, learning rate, and gamma. The y labels show the inference accuracy. The reward value of each plot indicates the reward criterion of the victim DRL model.

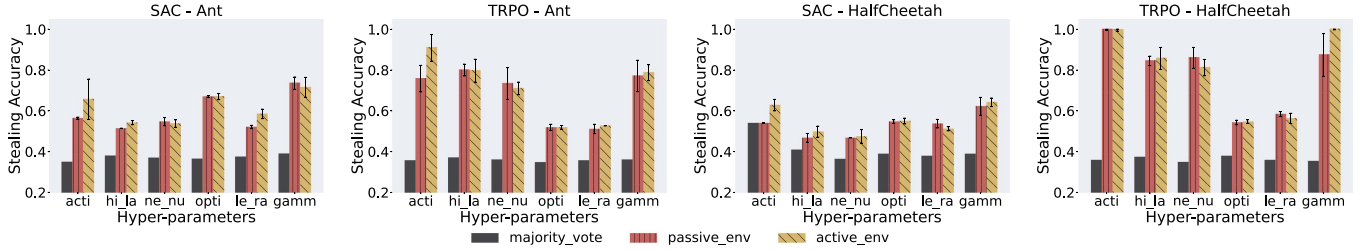


Fig. 4. The inference accuracy of different methods on four combinations of the DRL models (SAC and TRPO) and the environments (Ant and HalfCheetah). The x labels of each plot correspond in turn to the hyper-parameters activation function, hidden layers, neuron number, optimizer, learning rate, and gamma. The y labels show the inference accuracy. The reward value of each plot indicates the reward criterion of the victim DRL model.

C. Scalability on More Complex Cases

In this section, we conduct the inference attack against two advanced DRL models (SAC and TRPO) on two more complex tasks (HalfCheetah and Ant). Fig. 4 demonstrates HyperInfer can still perform well in these cases. HyperInfer infer the hyper-parameter settings based on inconsistent actions. Since DRL policy networks are often sub-optimal in practice, it is possible to infer hyper-parameters.

HyperInfer can alleviate the cost of training shadow models by reducing the number of shadow models required for each hyper-parameter setting. Based on the ablation study in Appendix D, available online, the active method can still achieve competitive inference accuracy with only 4 shadow models for each hyper-parameter setting. In future work, the attacker can collect more sequences for a shadow model or use a stronger classification model to further reduce the attack cost.

D. Possible Defense

To mitigate the risk of hyperparameter inference and protect the configuration of DRL models from auditing attacks, we investigate two practical defense strategies tailored to different setting of the action space. For discrete action environments, we restrict the output information by publishing only the predicted

actions without associated confidence scores. For continuous action tasks, we adopt local differential privacy mechanisms to perturb the released actions, aiming to obscure the underlying training configuration.

Releasing only actions: We reduce the information accessible to the adversary by releasing only the predicted action instead of the confidence values (posteriors) and empirically evaluate its effectiveness. For discrete action tasks (i.e., CartPole and Acrobot), directly publishing predicted actions without confidence scores is a strong protection method, considering the perturbed confidence scores usually do not affect the choice of predicted actions in practice.

Fig. 5 depicts the impact of the defense approach with or without. The inference accuracy of HyperInfer decreases (remaining superior to the baseline method) when only recording the predicted action, e.g., the inference accuracy is still 15% higher than the baseline with the active inference attack methods in the optimizer inference of PPO. Following Appendix D, available online, the active method can distill more information about the hyper-parameter setting from the limited predictions, making it more resistant to the defense approach.

Differential privacy: For the continuous action tasks, i.e., Ant and HalfCheetah, we perturb the DRL model's actions under different privacy (DP) guarantees to protect the

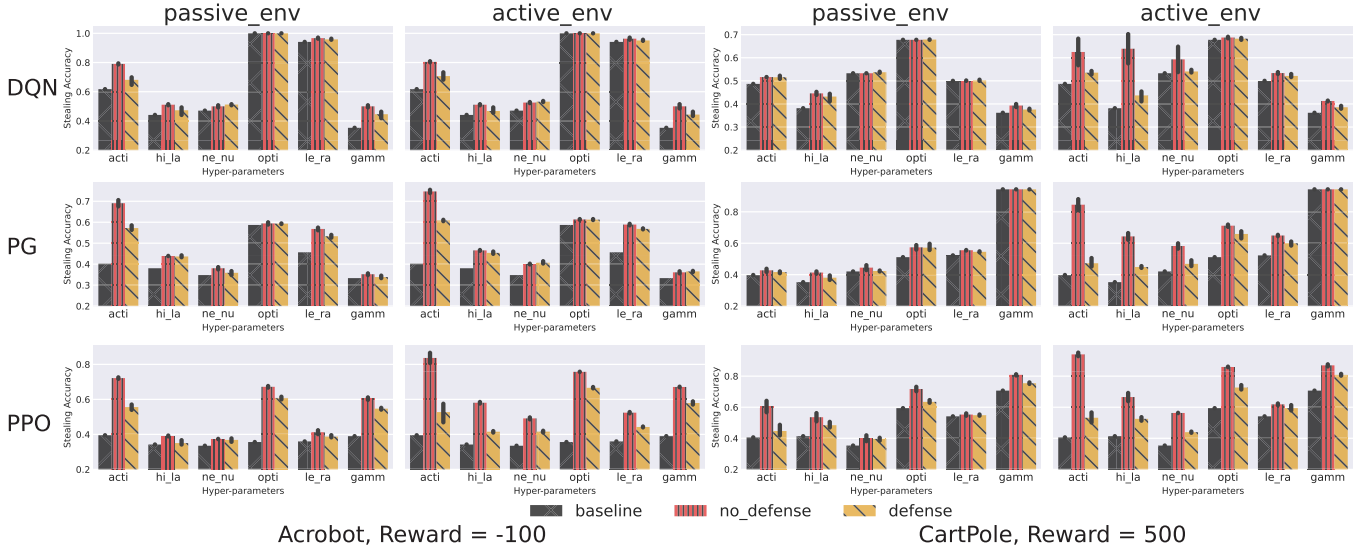


Fig. 5. The inference accuracy of different methods against the defense approach. In each subplot, the “baseline” is the “majority_vote” method, and “defense” means the adversary only obtains the predicted action without the confidence score.

TABLE II

THE INFERENCE ACCURACY OF HyperInfer AGAINST THE DP PERTURBATION. THE “MAJORITY” IS THE “MAJORITY VOTE” METHOD. THE “PASSIVE_ENV (DP GAUSSIAN)” AND “PASSIVE_ENV (DP LAPLACIAN)” COLUMNS SHOW THE CHANGES IN THE INFERENCE ACCURACY UNDER DP PERTURBATION, COMPARED WITH THOSE IN THE “PASSIVE_ENV” COLUMN. THE “ACTIVE_ENV (DP GAUSSIAN)” AND “ACTIVE_ENV (DP LAPLACIAN)” COLUMNS SHOW THE CHANGES IN THE INFERENCE ACCURACY UNDER DP PERTURBATION, COMPARED WITH THOSE IN THE “ACTIVE_ENV” COLUMN.

Task	Model	HyPara	majority	passive_env	passive_env (DP Gaussian)	passive_env (DP Laplacian)	active_env	active_env (DP Gaussian)	active_env (DP Laplacian)
Ant	SAC	acti	0.351	0.564±0.005	0.385±0.023	0.407±0.027	0.657±0.099	0.405±0.014	0.430±0.015
		hi_la	0.381	0.514±0.001	0.402±0.020	0.412±0.021	0.542±0.010	0.415±0.022	0.429±0.019
		ne_nu	0.371	0.547±0.020	0.386±0.025	0.424±0.037	0.538±0.018	0.396±0.032	0.458±0.021
		opti	0.366	0.669±0.004	0.420±0.052	0.465±0.065	0.670±0.015	0.468±0.030	0.530±0.005
		le_ra	0.376	0.521±0.008	0.393±0.017	0.419±0.034	0.585±0.023	0.402±0.020	0.453±0.010
		gamm	0.391	0.736±0.030	0.398±0.007	0.513±0.044	0.715±0.049	0.404±0.006	0.551±0.029
	TRPO	acti	0.358	0.758±0.065	0.473±0.129	0.606±0.137	0.909±0.065	0.575±0.111	0.738±0.032
		hi_la	0.372	0.800±0.029	0.407±0.047	0.535±0.077	0.797±0.056	0.430±0.057	0.611±0.009
		ne_nu	0.362	0.734±0.078	0.402±0.057	0.515±0.086	0.710±0.031	0.432±0.068	0.600±0.014
		opti	0.349	0.519±0.015	0.381±0.017	0.420±0.047	0.519±0.009	0.396±0.009	0.464±0.021
Half.	SAC	le_ra	0.358	0.511±0.023	0.379±0.013	0.422±0.050	0.527±0.001	0.390±0.007	0.472±0.009
		gamm	0.362	0.771±0.077	0.403±0.040	0.568±0.140	0.787±0.039	0.424±0.048	0.705±0.020
		acti	0.54	0.540±0.001	0.540±0.000	0.540±0.001	0.628±0.028	0.540±0.001	0.540±0.000
		hi_la	0.41	0.468±0.021	0.410±0.000	0.410±0.000	0.498±0.027	0.410±0.000	0.410±0.000
		ne_nu	0.365	0.468±0.001	0.370±0.004	0.372±0.008	0.475±0.033	0.371±0.005	0.365±0.001
		opti	0.39	0.547±0.009	0.396±0.004	0.403±0.010	0.550±0.014	0.397±0.006	0.394±0.004
	TRPO	le_ra	0.38	0.537±0.021	0.390±0.006	0.393±0.007	0.512±0.010	0.389±0.006	0.389±0.002
		gamm	0.39	0.622±0.044	0.398±0.006	0.445±0.056	0.642±0.021	0.396±0.006	0.391±0.001
		acti	0.36	0.998±0.002	0.559±0.124	0.747±0.079	0.996±0.005	0.668±0.085	0.798±0.028
		hi_la	0.375	0.845±0.023	0.430±0.089	0.598±0.059	0.857±0.054	0.479±0.105	0.635±0.057
Half.	SAC	ne_nu	0.35	0.860±0.051	0.506±0.095	0.608±0.090	0.812±0.040	0.583±0.042	0.676±0.020
		opti	0.38	0.543±0.011	0.384±0.005	0.445±0.031	0.547±0.009	0.382±0.004	0.467±0.021
		le_ra	0.36	0.584±0.012	0.399±0.025	0.441±0.015	0.563±0.025	0.423±0.008	0.452±0.008
		gamm	0.355	0.874±0.105	0.627±0.135	0.668±0.080	0.999±0.002	0.759±0.027	0.724±0.054
	TRPO	acti	0.36	0.998±0.002	0.559±0.124	0.747±0.079	0.996±0.005	0.668±0.085	0.798±0.028
		hi_la	0.375	0.845±0.023	0.430±0.089	0.598±0.059	0.857±0.054	0.479±0.105	0.635±0.057
		ne_nu	0.35	0.860±0.051	0.506±0.095	0.608±0.090	0.812±0.040	0.583±0.042	0.676±0.020
		opti	0.38	0.543±0.011	0.384±0.005	0.445±0.031	0.547±0.009	0.382±0.004	0.467±0.021
	SAC	le_ra	0.36	0.584±0.012	0.399±0.025	0.441±0.015	0.563±0.025	0.423±0.008	0.452±0.008
		gamm	0.355	0.874±0.105	0.627±0.135	0.668±0.080	0.999±0.002	0.759±0.027	0.724±0.054

hyper-parameters of the DRL model. More specifically, the Ant task has an 8-dimensional continuous action space, while HalfCheetah has 6 dimensions. Each component is bounded within $[-1, 1]$, the global ℓ_2 -sensitivity is $\sqrt{8} \approx 2.828$ for Ant and $\sqrt{6} \approx 2.449$ for HalfCheetah. Under the Laplace mechanism, the noise scale b is computed as $b = \Delta/\epsilon$, yielding 2.828 and 2.449, respectively, when $\epsilon = 1$. For the Gaussian mechanism with privacy parameter ($\epsilon = 1, \delta = 10^{-5}$), the standard

deviation σ must satisfy $\sigma \geq \frac{\sqrt{2\ln(1.25/\delta)} \cdot \Delta}{\epsilon}$. This results in $\sigma \approx 13.71$ for Ant and $\sigma \approx 11.87$ for HalfCheetah.

Tables II and III demonstrate that although local DP perturbations effectively preserve the privacy of DRL hyper-parameters by reducing the inference accuracy of the auditing methods, they simultaneously induce noticeable performance degradation. Specifically, as presented in Table II, added noise causes

TABLE III

THE PERFORMANCE DECAY OF THE DRL AGENTS UNDER DP PERTURBATION. THE “PERFORMANCE” COLUMN SHOWS THE ORIGINAL PERFORMANCE OF THE DRL AGENTS. THE “PERFORMANCE (DP GAUSSIAN)” AND “PERFORMANCE (DP LAPLACIAN)” COLUMNS SHOW THE DRL AGENTS’ PERFORMANCE UNDER DP.

Task	Model	Performance	Performance (DP Gaussian)	Performance (DP Laplacian)
Ant	SAC	813.0 $\pm$ 71.6	-13.6 $\pm$ 3.3	-12.0 $\pm$ 3.3
	TRPO	629.0 $\pm$ 136.1	-14.9 $\pm$ 3.5	-14.0 $\pm$ 3.9
HalfCheetah	SAC	557.8 $\pm$ 55.3	-525.9 $\pm$ 50.7	-381.3 $\pm$ 50.8
	TRPO	677.5 $\pm$ 77.0	-520.2 $\pm$ 44.4	-386.5 $\pm$ 31.0

a significant drop in inference accuracy in both passive and active environments. For instance, in the Ant environment under SAC, the inference accuracy decreases by over 0.14 when using Gaussian noise and by over 0.13 with Laplacian noise, compared to the non-private setting. From Table III, SAC and TRPO agents suffer extreme performance drops in the HalfCheetah task, more than 1000 points under Gaussian noise and nearly 1200 under Laplacian noise. The injected noise severely hinders the agent’s ability to learn and act optimally. While DP perturbation helps defend against hyperparameter leakage, it also introduces notable overhead in terms of policy performance.

E. Takeaways

Through the extensive experiments, we have obtained the following answers to the starting questions of Section V:

- A1. The inference accuracy of HyperInfer is higher than the baseline method in all the evaluated scenarios, which shows that the behaviors of the DRL models can indeed expose the hyper-parameter settings (Section V-B).
- A2. The states optimized by the active inference attack method further utilize the gradients of the shadow models, which obtain a better inference accuracy (Section V).
- A3. The DRL models with higher reward criterion suffer more privacy risk against HyperInfer (Appendix E), available online. The hyper-parameter has various influence strength on the action preference (Appendix F), available online. HyperInfer benefits from a larger length of state-action sequence or the number of shadow models (Appendix D), available online.
- A4. The inference accuracy of HyperInfer decreases (remaining superior to the baseline method) when only recording the predicted action (Section V-D).

VI. CASE STUDY

From Section V, we illustrate that HyperInfer can violate the DRL model’s intellectual property (IP), i.e., inferring the hyper-parameter settings by the closed-box access. In this section, we demonstrate how the knowledge of hyperparameter can enhance existing attacks. In practice, the disclosed information will make the model more vulnerable to attacks from adversaries, such as adversarial attacks [20], [36], [44], model extraction attacks [5], [9], [56], [87], and adversarial policy attacks [6], [28], [34], [38].

Adversarial attacks: We randomly select 20 victim models with different hyper-parameter settings and generate 20×8 shadow models with or without knowing the hyper-parameters

TABLE IV

THE PERCENTAGE OF SHADOW AGENTS WITH AVERAGE TRANSFERABILITY VALUES MORE THAN 90%. THE RESULTS ARE COLLECTED FROM PPO AND THE CARTPOLE ENVIRONMENT.

Reward=300								
ϵ	FGM		L2PGD		FGSM		LinfPGD	
	w.	w/o	w.	w/o	w.	w/o	w.	w/o
1e-4	20.62	15.62	13.12	13.75	24.38	16.88	19.38	15.00
1e-3	51.25	43.75	50.62	43.75	51.88	44.38	50.62	44.38
5e-3	51.88	44.38	51.25	44.38	51.88	44.38	51.25	44.38
1e-2	51.88	44.38	51.25	44.38	51.88	44.38	51.25	44.38
1e-1	51.88	44.38	46.88	41.88	51.88	44.38	50.00	42.50

Reward=500								
ϵ	FGM		L2PGD		FGSM		LinfPGD	
	w.	w/o	w.	w/o	w.	w/o	w.	w/o
1e-4	10.00	9.38	5.62	5.00	11.88	11.25	9.38	8.75
1e-3	58.13	37.50	56.25	36.88	60.00	38.12	58.75	38.12
5e-3	61.25	39.38	61.25	40.00	61.25	39.38	61.25	39.38
1e-2	61.25	39.38	60.62	39.38	61.25	40.00	60.62	39.38
1e-1	61.25	40.00	58.75	37.50	61.25	40.00	58.75	37.50

separately. For each white-box shadow model, we generate a 500-step adversarial state, apply the adversarial state to the victim model and record the attack success rate, i.e., transferability value. We repeat the transferability experiment 10 times and obtain the average values for each shadow model. We count the percentage of shadow agents in all 20×8 shadow models with average transferability values more than 90%. Since GAIL [25] adopts the actor-critic framework as the generator, we use the PPO agents as victim models to eliminate the influence of the DRL framework. We show the results of four adversarial example generation methods, i.e., FGM, FGSM [20], L2PGD and LinfPGD [36], [44]. These methods are widely adopted and canonical in evaluating the robustness of deep reinforcement learning agents under adversarial perturbations. Previous work such as Huang et al. [27] and Pattanaik et al. [52] used similar settings to assess DRL robustness.

Table IV shows the percentage of shadow agents with high average transferability values. We set five different ϵ values to control the strength of an adversarial attack. The ‘W’ and ‘W/O’ illustrate the transferability experiment with or without the hyper-parameters information. From the results, we have the following observations. 1) The inference attack improves significantly in the downstream attacks. When the shadow models use the same hyper-parameter settings as the victim models, their decision boundaries tend to be closed during the training process. Thus, adversarial transferability has more than 15% boost in most cases. 2) By comparing the results in Table IV, the enhancement in transferability becomes more obvious for DRL agents with a higher reward criterion, which is consistent with the analysis in Appendix E, available online. We speculate that the agents with higher reward criteria have more convergent decision boundaries, and hyper-parameters have more influence on the formation of their decision boundaries. 3) The transferability values are saturated as the epsilon increases. The environments interacting with the DRL agents have their physical constraints. For example, the state of the “CartPole” environment is defined by four physical variables, i.e., cart position, cart velocity, pole angle, and pole angular velocity, ranging from $[-2.4, \infty, 12 \text{ deg}, \infty]$ to $[+2.4, \infty, 12 \text{ deg}, \infty]$. When one value

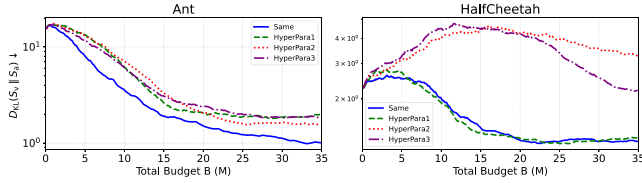


Fig. 6. Model extraction attacks against the DRL model. The Kullback-Leibler (KL) divergence measures the discrepancy between the adversarial model and the victim model [lower is better]. “Same” means the adversarial model shares the hyperparameter settings of the victim model. “HyperParaX” means the adversarial model has different degrees of hyperparameter deviation with the victim model.

of the state exceeds the value range, the generation of adversarial states should stop to satisfy the physical constraints. Therefore, the transferability values cannot always increase with larger ϵ .

Model extraction attack: Several prior studies [5], [9], [56], [85], [87] have proposed model extraction attacks targeting DRL models. In our evaluation, we select the Stealthy Imitation by Zhuang et al. [87], which represents the state-of-the-art in policy stealing for DRL. We conduct experiments on both Ant and HalfCheetah tasks following the methodology of [87]. We use the Kullback–Leibler (KL) divergence to quantify the discrepancy between the adversarial model and the victim model. As shown in Fig. 6, the KL divergence tends to decrease more rapidly and reach lower values when the adversarial model shares the same hyper-parameters as the victim model.

In the “HyperPara1” setting, the adversarial model adopts a different discount factor, changing γ from 0.99 to 0.9. Based on this, “HyperPara2” introduces an additional architectural deviation by reducing the number of hidden neurons from 256 to 64. “HyperPara3” represents the most divergent setting, where γ is set to 0.9, the size of the hidden layer is reduced to 64 neurons, and the learning rate is reduced from 0.001 to 0.0001. The results demonstrate that larger hyperparameter discrepancies lead to larger divergences between adversarial and victim models.

Adversarial policy attack: We adopt the adversarial policy attack proposed by Gleave et al. [17], which is a canonical method in this line of work and has inspired several follow-up studies [21], [41], [43]. Our experiments are carried out on the SumoHumans task, where we follow the evaluation protocol of [17], reporting win rates as the adversarial policy is trained against the victim model.

As shown in Fig. 7, the adversarial agent achieves higher and faster win rates when its hyper-parameters match the victim’s. In the “HyperPara1” setting, the adversarial model’s learning rate is increased from 3×10^{-4} to 3×10^{-3} . “HyperPara2” increases the learning rate to 3×10^{-2} , representing a more aggressive deviation. The experimental results suggest that increasing the hyperparameter mismatch weakens the effectiveness of the adversarial policy attack.

VII. RELATED WORK

Privacy attacks against DRL: Privacy threats in DRL can be broadly classified into model privacy, reward privacy, and environment privacy attacks [37], [48].

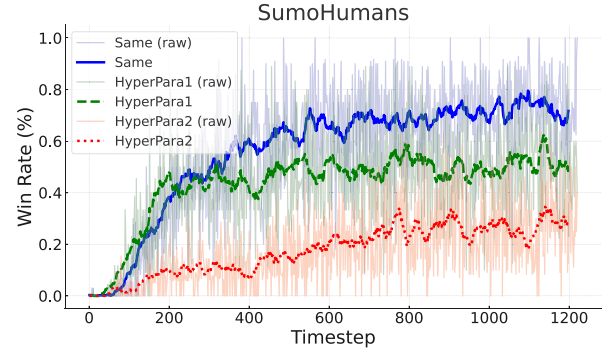


Fig. 7. Adversarial policy Attack against the DRL model. The Y-axis shows the win rate while training the adversary model against the victim model in the SumoHumans environment [higher is better]. “Same” means the adversarial model shares the hyperparameter settings of the victim model. “HyperParaX” means the adversarial model has different degrees of hyperparameter deviation from the victim model. The “HyperParaX” plots represent smoothed demonstrations of the “HyperParaX (raw)” data.

Model privacy attacks aim to recover internal policies or training settings. A primary class of such attacks is model extraction. Behzadan et al. [5] showed that adversaries can replicate policy behavior via imitation. Chen et al. [9] further investigated the feasibility and potential gains of stealing DRL models. Later works enhanced stealth and practicality. Sajid et al. [56] analyzed extraction in controller-based systems, while Zhuang et al. [87] introduced an environment-free, reward-guided method. Notably, HyperInfer complements this line by being the first to infer DRL hyper-parameters as a novel form of model privacy breach. *Reward privacy attacks* reconstruct the agent’s reward function via inverse reinforcement learning (IRL), risking disclosure of sensitive design intents [10], [67]. For instance, Prakash et al. [54] proposed PRIL to quantify the vulnerability of differentially private RL models to such inference. *Environment privacy attacks* infer hidden environment information from the model’s behavior. Pan et al. [51] highlighted this threat in robot navigation. Prakash et al. [54] applied membership inference to determine if particular data points were used in training. Gomrokchi et al. [18] further showed that observation leakage is exacerbated under partial observability.

Other attacks against DRL: Beyond privacy, DRL systems are vulnerable to adversarial attacks, adversarial policies, and backdoor attacks. *Adversarial attacks* perturb inputs to mislead DRL agents. Huang et al. [27] introduced uniform attacks that apply perturbations at every timestep. Lin et al. [39] improved efficiency via strategically-timed perturbations. Sun et al. [63] addressed temporal dependencies by proposing critical-point attacks that degrade performance more effectively. *Adversarial policies* exploit opponent weaknesses in competitive RL. Gleave et al. [17] showed agents can exhibit unnatural yet effective behaviors to defeat stronger opponents. Guo et al. [21] introduced non-zero-sum attacks that simultaneously boost attacker rewards and harm victims. Even with partial observability or limited control, adversarial agents remain a threat [41], [43]. *Backdoor attacks* inject triggers during training to induce malicious behaviors at test time. TrojDRL [32] overlays visual triggers and manipulates rewards to plant target actions.

Ashcraft et al. [4] improved stealth by using in-distribution triggers. Yu et al. [83] proposed temporal-pattern backdoors tailored to partially observable settings.

Privacy attacks and defenses in other DNNs: Privacy attacks in traditional DNNs fall into four categories [55]: (1) *Membership inference* detects whether a sample was in the training set [26], [40], [60], [70]; (2) *Reconstruction attacks* aim to recover training samples or features [19], [57], [73], [86]; (3) *Property inference* extracts latent dataset-level attributes [15], [29], [45], [62]; (4) *Model extraction* builds surrogate models to replicate decision boundaries [31], [35], [50], [64]. To mitigate these privacy risks, various defenses have been developed. SI-Guard [70] adds noise to encrypted outputs in MPC inference to resist membership attacks, while Leia [42] protects inputs and model parameters via lightweight cryptography for edge devices. Emerging domains have also adopted privacy-preserving techniques. GraphGuard [77] enables data-free membership detection and unlearning in GNNs. OblivGNN [80] secures inference and updates via secret sharing. For LLMs, Self-Guard [74] empowers models to detect unsafe outputs during inference, and Uzor et al. [65] proposed a local gatekeeper to filter sensitive inputs before cloud transmission.

VIII. CONCLUSION

In this work, we propose a novel hyper-parameter inference method against the DRL model by leveraging the state-action records from the victim model. HyperInfer achieves good inference performance with varying DRL model and environment combinations. Through the experimental evaluation, an attacker can infer the hyper-parameter settings by collecting the behavior of the DRL model. By studying various parameter settings, we conclude several important observations for the DRL practitioners. The relationship between the performance and the privacy risk of the DRL models reminds the DRL practitioners to consider privacy leakage risk when pursuing high performance. Finally, we compare the effectiveness of the existing ML attacks against closed-box DRL models with or without the hyper-parameter information to demonstrate the negative impact of hyper-parameter leakage.

REFERENCES

- [1] S. Amarjyoti, "Deep reinforcement learning for robotic manipulation - The state of the art," 2017, *arXiv: 1701.08878*.
- [2] Y. Amitai and O. Amir, "I don't think so: Disagreement-based policy summaries for comparing agents," 2021, *arXiv:2102.03064*.
- [3] K. Arulkumar, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Process. Mag.*, vol. 34, no. 6, pp. 26–38, Nov. 2017.
- [4] C. Ashcraft and K. Karra, "Poisoning deep reinforcement learning agents with in-distribution Triggers," 2021, *arXiv:2106.07798*.
- [5] V. Behzadan and A. M. Hsu, "Adversarial exploitation of policy imitation," 2019, *arXiv: 1906.01121*.
- [6] V. Behzadan and A. Munir, "Vulnerability of deep reinforcement learning to policy induction attacks," in *Proc. Int. Conf. Mach. Learn. Data Mining Pattern Recognit.*, 2017, pp. 262–275.
- [7] C. Berner et al., "Dota 2 with large scale deep reinforcement learning," 2019, *arXiv: 1912.06680*.
- [8] K. Chen et al., "Stealing deep reinforcement learning models for fun and profit," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2021, pp. 307–319.
- [9] K. Chen, S. Guo, T. Zhang, X. Xie, and Y. Liu, "Stealing deep reinforcement learning models for fun and profit," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2021, pp. 307–319.
- [10] S. R. Chirra, P. Varakantham, and P. Paruchuri, "Preserving the privacy of reward functions in MDPs through deception," 2024, *arXiv:2407.09809*.
- [11] Y. Deng, F. Bao, Y. Kong, Z. Ren, and Q. Dai, "Deep direct reinforcement learning for financial signal representation and trading," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 3, pp. 653–664, Mar. 2017.
- [12] P. Dhariwal et al., "OpenAI baselines," 2017. [Online]. Available: <https://github.com/openai/baselines>
- [13] A. R. Fayjie, S. Hossain, D. Oualid, and D.-J. Lee, "Driverless car: Autonomous driving using deep reinforcement learning in urban environment," in *Proc. Int. Conf. Ubiquitous Robots*, 2018, pp. 896–901.
- [14] J. Gajcin, R. Nair, T. Pedapati, R. Marinescu, E. Daly, and I. Dusparic, "Contrastive explanations for comparing preferences of reinforcement learning agents," 2021, *arXiv:2112.09462*.
- [15] K. Ganju, Q. Wang, W. Yang, C. A. Gunter, and N. Borisov, "Property inference attacks on fully connected neural networks using permutation invariant representations," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 619–633.
- [16] A. Gleave et al., "Adversarial policies: Attacking deep reinforcement learning," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–16.
- [17] A. Gleave, M. Dennis, C. Wild, N. Kant, S. Levine, and S. Russell, "Adversarial policies: Attacking deep reinforcement learning," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–16.
- [18] M. Gomrokchi, S. Amin, H. Aboutalebi, A. Wong, and D. Precup, "Membership inference attacks against temporally correlated data in deep reinforcement learning," *IEEE Access*, vol. 11, pp. 42796–42808, 2023.
- [19] N. Z. Gong and B. Liu, "You are who you know and how you behave: Attribute inference attacks via users' social friends and behaviors," in *Proc. USENIX Conf. Secur. Symp.*, 2016, pp. 979–995.
- [20] I. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proc. Int. Conf. Learn. Representations*, 2015, pp. 1–11.
- [21] W. Guo, X. Wu, S. Huang, and X. Xing, "Adversarial policy learning in two-player competitive games," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 3910–3919.
- [22] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2018, pp. 1861–1870.
- [23] N. Heess et al., "Emergence of locomotion behaviours in rich environments," 2017, *arXiv: 1707.02286*.
- [24] A. Hill et al., "Stable baselines," 2018. [Online]. Available: <https://github.com/hill-a/stable-baselines>
- [25] J. Ho and S. Ermon, "Generative adversarial imitation learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2016, pp. 4572–4580.
- [26] Y. Hu et al., "Membership inference attacks against vision-language models," 2025, *arXiv:2501.18624*.
- [27] S. Huang, N. Papernot, I. Goodfellow, Y. Duan, and P. Abbeel, "Adversarial attacks on neural network policies," 2017, *arXiv: 1702.02284*.
- [28] S. H. Huang et al., "Adversarial attacks on neural network policies," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–7.
- [29] Y. Huang, Z. Zhang, Q. Zhao, X. Yuan, and C. Chen, "THEMIS: Towards practical intellectual property protection for post-deployment on-device deep learning models," in *Proc. USENIX Conf. Secur. Symp.*, 2025, pp. 7311–7330.
- [30] I. Ilahi et al., "Challenges and countermeasures for adversarial attacks on deep reinforcement learning," 2020, *arXiv: 2001.09684*.
- [31] M. Jagielski, N. Carlini, D. Berthelot, A. Kurakin, and N. Papernot, "High accuracy and high fidelity extraction of neural networks," in *Proc. USENIX Conf. Secur. Symp.*, 2020, pp. 1345–1362.
- [32] P. Kiourt, K. Wardega, S. Jha, and W. Li, "TrojDRL: Evaluation of backdoor attacks on deep reinforcement learning," in *Proc. ACM/IEEE Des. Autom. Conf.*, 2020, pp. 1–6.
- [33] V. R. Konda and J. N. Tsitsiklis, "Actor-critic algorithms," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 1999, pp. 1008–1014.
- [34] J. Kos and D. Song, "Delving into adversarial attacks on deep policies," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–6.
- [35] K. Krishna, G. S. Tomar, A. P. Parikh, N. Papernot, and M. Iyyer, "Thieves on sesame street! model extraction of BERT-based APIs," 2019, *arXiv: 1910.12366*.
- [36] A. Kurakin, I. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," 2016, *arXiv:1607.02533*.

- [37] Y. Lei, D. Ye, S. Shen, Y. Sui, T. Zhu, and W. Zhou, "New challenges in reinforcement learning: A survey of security and privacy," *Artif. Intell. Rev.*, vol. 56, no. 7, pp. 7195–7236, 2023.
- [38] Y. Lin et al., "Tactics of adversarial attack on deep reinforcement learning agents," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–5.
- [39] Y.-C. Lin, Z.-W. Hong, Y.-H. Liao, M.-L. Shih, M.-Y. Liu, and M. Sun, "Tactics of adversarial attack on deep reinforcement learning agents," in *Proc. Int. Joint Conf. Artif. Intell.*, 2017, pp. 3756–3762.
- [40] L. Liu, Y. Wang, G. Liu, K. Peng, and C. Wang, "Membership inference attacks against machine learning models via prediction sensitivity," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 3, pp. 2341–2347, May/Jun. 2023.
- [41] X. Liu, S. Chakraborty, Y. Sun, and F. Huang, "Rethinking adversarial policies: A generalized attack formulation and provable defense in RL," 2023, *arXiv:2305.17342*.
- [42] X. Liu, B. Wu, X. Yuan, and X. Yi, "Leia: A lightweight cryptographic neural network inference system at the edge," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 237–252, 2021.
- [43] O. Ma et al., "SUB-PLAY: Adversarial policies against partially observed multi-agent reinforcement learning systems," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2024, pp. 645–659.
- [44] A. Madry et al., "Towards deep learning models resistant to adversarial attacks," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–23.
- [45] S. Mahloujifar, E. Ghosh, and M. Chase, "Property inference from poisoning," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 1120–1137.
- [46] V. Mnih et al., "Asynchronous methods for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1928–1937.
- [47] V. Mnih et al., "Playing atari with deep reinforcement learning," 2013, *arXiv:1312.5602*.
- [48] K. Mo, P. Ye, X. Ren, S. Wang, W. Li, and J. Li, "Security and privacy issues in deep reinforcement learning: Threats and countermeasures," *ACM Comput. Surv.*, vol. 56, no. 6, pp. 1–39, 2024.
- [49] S. J. Oh et al., "Towards reverse-engineering black-box neural networks," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–20.
- [50] T. Orekondy, B. Schiele, and M. Fritz, "Knockoff nets: Stealing functionality of black-box models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 4954–4963.
- [51] X. Pan et al., "How you act tells a lot: Privacy-leaking attack on deep reinforcement learning," in *Proc. Int. Conf. Auton. Agents MultiAgent Syst.*, 2019, pp. 368–376.
- [52] A. Pattanaik et al., "Robust deep reinforcement learning with adversarial attacks," in *Proc. Int. Conf. Auton. Agents MultiAgent Syst.*, 2018, pp. 2040–2042.
- [53] I. P. Pavlov and W. H. Gantt, *Conditioned Reflexes and Psychiatry*, vol. 2. New York, NY, USA: International Publishers, 1941.
- [54] K. Prakash et al., "How private is your RL policy? An inverse RL based analysis framework," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 8009–8016.
- [55] M. Rigaki and S. Garcia, "A survey of privacy attacks in machine learning," *ACM Comput. Surv.*, vol. 56, no. 4, pp. 1–34, 2023.
- [56] M. Sajid, Y. Shen, and Y. Shoukry, "Model extraction attacks against reinforcement learning based controllers," in *IEEE Conf. Decis. Control*, 2023, pp. 813–820.
- [57] A. Salem, A. Bhattacharya, M. Backes, M. Fritz, and Y. Zhang, "Updates-leak: Data set inference and reconstruction attacks in online learning," in *Proc. USENIX Conf. Secur. Symp.*, 2020, pp. 1291–1308.
- [58] J. Schulman et al., "Trust region policy optimization," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1889–1897.
- [59] J. Schulman et al., "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [60] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 3–18.
- [61] D. Silver et al., "Mastering the game of go without human knowledge," *Nature*, vol. 550, pp. 354–359, 2017.
- [62] C. Song and A. Raghunathan, "Information leakage in embedding models," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 377–390.
- [63] J. Sun et al., "Stealthy and efficient adversarial attacks against deep reinforcement learning," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 5883–5891.
- [64] F. Tramèr et al., "Stealing machine learning models via prediction APIs," in *Proc. USENIX Conf. Secur. Symp.*, 2016, pp. 601–618.
- [65] G. Uzor, H. Al-Qudah, Y. Ineza, and A. Serwadda, "Guarding your conversations: Privacy gatekeepers for secure interactions with cloud-based ai models," in *Proc. IEEE Int. Conf. Semantic Comput.*, 2025, pp. 235–242.
- [66] B. Wang and N. Z. Gong, "Stealing hyperparameters in machine learning," in *Proc. IEEE Symp. Secur. Privacy*, 2018, pp. 36–52.
- [67] B. Wang and N. Hegde, "Privacy-preserving Q-learning with functional noise in continuous spaces," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 11323–11333.
- [68] L. Wang et al., "BACKDOORL: Backdoor attack against competitive reinforcement learning," in *Proc. Int. Joint Conf. Artif. Intell.*, 2021, pp. 3699–3705.
- [69] S. Wang, "Security and ownership verification in deep reinforcement learning," Master's thesis, Cheriton School Comput. Sci., Univ. Waterloo, Waterloo, Canada, 2022.
- [70] X. Wang, X. Liu, S. Lai, X. Yi, and X. Yuan, "SIGuard: Guarding secure inference with post data privacy," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2025, pp. 1–18.
- [71] X. Wang et al., "Deep reinforcement learning: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 4, pp. 5064–5078, Apr. 2024.
- [72] Y. Wang, E. Sarkar, W. Li, M. Maniatakos, and S. E. Jabari, "Stop-and-Go: Exploring backdoor attacks on deep reinforcement learning-based traffic congestion control systems," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4772–4787, 2021.
- [73] Z. Wang, M. Song, Z. Zhang, Y. Song, Q. Wang, and H. Qi, "Beyond inferring class representatives: User-level privacy leakage from federated learning," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 2512–2520.
- [74] Z. Wang et al., "SELF-GUARD: Empower the LLM to safeguard itself," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2024, pp. 1648–1668.
- [75] C. J. C. H. Watkins, "Learning from delayed rewards," PhD thesis, King's College, Cambridge Univ., Cambridge, England, 1989.
- [76] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Mach. Learn.*, vol. 8, pp. 229–256, 1992.
- [77] B. Wu et al., "GraphGuard: Detecting and counteracting training data misuse in graph neural networks," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2025, pp. 1–20.
- [78] X. Wu et al., "Adversarial policy training against deep reinforcement learning," in *Proc. USENIX Conf. Secur. Symp.*, 2021, pp. 1883–1900.
- [79] T. Xu et al., "Learning to explore via meta-policy gradient," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5459–5468.
- [80] Z. Xu et al., "OblivGNN: Oblivious inference on transductive and inductive graph neural network," in *Proc. USENIX Conf. Secur. Symp.*, 2024, pp. 2209–2226.
- [81] M. Xue, Y. Zhang, J. Wang, and W. Liu, "Intellectual property protection for deep learning models: Taxonomy, methods, attacks, and evaluations," *IEEE Trans. Artif. Intell.*, vol. 3, no. 6, pp. 908–923, Dec. 2022.
- [82] Z. Yang, N. Iyer, J. Reimann, and N. Virani, "Design of intentional backdoors in sequential models," 2019, *arXiv:1902.09972*.
- [83] Y. Yu, J. Liu, S. Li, K. Huang, and X. Feng, "A temporal-pattern backdoor attack to deep reinforcement learning," in *Proc. IEEE Glob. Commun. Conf.*, 2022, pp. 2710–2715.
- [84] M. Zhang, Z. McCarthy, C. Finn, S. Levine, and P. Abbeel, "Learning deep neural network policies with continuous memory states," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2016, pp. 520–527.
- [85] X. Zhang, C. Fang, and J. Shi, "Thief, beware of what get you there: Towards understanding model extraction attack," 2021, *arXiv:2104.05921*.
- [86] Y. Zhang, R. Jia, H. Pei, W. Wang, B. Li, and D. Song, "The secret revealer: Generative model-inversion attacks against deep neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 253–261.
- [87] Z. Zhuang, M.-I. Nicolae, and M. Fritz, "Stealthy imitation: Reward-guided environment-free policy stealing," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 62682–62706.



Linkang Du (Member, IEEE) received the BE and PhD degrees from Zhejiang University, in 2018 and 2023, respectively. He is currently an assistant professor with the School of Cyber Science and Engineering, Xi'an Jiaotong University, Xi'an, China. His research interests focus on data security and privacy issues in machine learning applications, including privacy protection and dataset copyright auditing. His vision is to protect the legitimate data rights and interests of every individual in the era of widespread artificial intelligence.



in the big four security conferences. He is the recipient of Distinguished Paper Award and Distinguished Reviewer Award with NDSS 2024.

Zhikun Zhang (Member, IEEE) received the PhD degree in control science and engineering from Zhejiang University, in 2019. He is a ZJU 100-Young professor with the College of Computer Science and Technology, Zhejiang University. Before joining Zhejiang University, he was a visiting assistant professor with Stanford University and Research Group leader with CISPA Helmholtz Center for Information Security, Germany. His research interest concentrates on data privacy, differential privacy, and trustworthy artificial intelligence. He has published more than 20 papers



Science Award. He serves/served as associate editor for *IEEE Transactions on Cloud Computing* and *IEEE Transactions on Control of Network Systems*. He also serves/served as a guest editor for *IEEE Transactions on Automatic Control* and *IEEE Transactions on Signal and Information Processing over Networks*.

Peng Cheng (Member, IEEE) received the BSc and PhD degrees in control science and engineering from Zhejiang University, Hangzhou, China, in 2004 and 2009, respectively. He is currently a professor and the dean of the College of Control Science and Engineering, Zhejiang University. His research interests include cyber-physical systems security, networked sensing and control, and cloud networking. He has been awarded the 2020 Changjiang Scholars chair professor. He has received the State Science and Technology Progress Award and the MOE Natural



Min Chen (Member, IEEE) received the PhD degree from the CISPA Helmholtz Center for Information Security in Germany. She is an assistant professor with Vrije Universiteit Amsterdam. Her research centers on trustworthy AI, focusing on privacy and security in machine learning. She has published more than 10 papers in leading security conferences, including IEEE S&P, ACM CCS, NDSS, and USENIX Security. She was awarded the Distinguished Paper Award with NDSS 2024.



He serves/served on the editorial boards of multiple IEEE Transactions, e.g., *IEEE Transactions on Control of Network Systems*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Industrial Electronics*, *IEEE Transactions on Fuzzy Systems* and the general co-chairs for multiple IEEE/ACM conference. He was a member of Fellow Evaluation Committee and an distinguished lecturer of IEEE VTS, a fellow of CAA. His research interests include industrial IoT, networked control, cyber security.

Jiming Chen (Fellow, IEEE) received the PhD degree in control science and engineering from Zhejiang University, in 2005. He was vice dean of Faculty of Information Technology, deputy director of the State Key laboratory of Industrial Control Technology, and director of Industrial Process Control, Zhejiang University. He was a visiting researcher with the University of Waterloo from 2008 to 2010. Now he is a chair professor with the Department of Control Science and Engineering, Zhejiang University, and also is the president of Hangzhou Dianzi University.



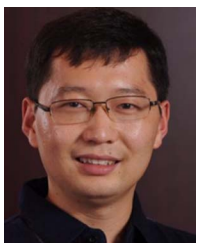
systems and cyber-physical energy system security and control.

Mingyang Sun (Senior Member, IEEE) received the PhD degree from the Department of Electrical and Electronic Engineering, Imperial College London, London, U.K., in 2017. From 2017 to 2019, he was a research associate and a DSI Affiliate fellow with Imperial College London, London, U.K. He is currently a research professor with the Department of Industrial Engineering and Management, College of Engineering, Peking University, Beijing, China. He is also an Honorary lecturer with Imperial College London. His research interests include AI in energy



ing Grant, the Caspar Bowden Privacy Award, the IEEE and ACM Fellowship, as well as various Career Awards and Best Paper Awards. In particular, He was the first German to receive the MIT TR35 Award. He is regularly listed in rankings as one of Germany's most important IT personalities.

Michael Backes (Fellow, IEEE) is the founding director and CEO of the CISPA Helmholtz Center for Information Security. His research results have shaped these scientific fields internationally, as is evidenced by more than 300 peer-reviewed and highly cited publications in renowned international journals and conference proceedings. His research has earned him internationally renowned scientific awards and honors, in particular the ERC Synergy Grant, which is Europe's most highly endowed EU group research award, the Karl Heinz Beckurts Prize, an ERC Start-



Membership chair of the IEEE Student Branch with Georgia State from 2012 to 2013.

Shouling Ji (Senior Member, IEEE) received the PhD degree in electrical and computer engineering from the Georgia Institute of Technology, and the PhD degree in computer science from Georgia State University. He is a ZJU 100-Young professor with the College of Computer Science and Technology, Zhejiang University, and a research faculty with the School of Electrical and Computer Engineering, Georgia Institute of Technology. His current research interests include AI security, data-driven security, privacy, and data analytics. He is a member of ACM. He was the



program committee of USENIX Security 2021, ACM CCS 2021 2020 2019, WWW 2021 2020, AAAI 2021, RAID 2020, ICWSM 2020, and PETS 2021 2020.

Yang Zhang (Member, IEEE) received the PhD degree from the University of Luxembourg, in November 2016. He is a faculty member with CISPA Helmholtz Center for Information Security, Germany. Previously, he was a group leader with CISPA. His research interests include the intersection of privacy and machine learning. Over the years, he has published multiple papers with top venues in computer science, including WWW, CCS, NDSS, and USENIX Security. His work has received NDSS 2019 Distinguished Paper Award. He has served in the technical